MCA Lab Assignments Compilation

Progressive Education Society's Modern College of Engineering | Python Implementations

Part 1: Basic Programming & Data Structures

1. Write a Python/R program to create three vectors a,b,c with 3 integers. Combine the three vectors to become a 3×3 matrix where each column represents a vector. Print the content of the matrix.

```
import numpy as np

# Create three vectors

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

c = np.array([7, 8, 9])

# Combine them column-wise

matrix = np.column_stack((a, b, c))

print("3x3 Matrix:\n", matrix)
```

2. Write a Python/R program to create a list containing a vector, a matrix and a list and give names to the elements in the list. Access the first and second element of the list.

```
import numpy as np

# Create a dictionary (Python's equivalent of a named list)

named_list = {

'my_vector': np.array([10, 20, 30]),

'my_matrix': np.array([[1, 2], [3, 4]]),

'my_list': [100, 200, 300]

}

# Access the first and second elements
```

3. **Write a Python/R program to create an array with three columns, three rows, and two "tables", taking two vectors as input to the array. Print the array.**

```
import numpy as np

# Create two input vectors

v1 = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])

v2 = np.array([10, 11, 12, 13, 14, 15, 16, 17, 18])

# Combine and reshape into (2 tables, 3 rows, 3 columns)

array_3d = np.concatenate((v1, v2)).reshape(2, 3, 3)

print("3D Array (2 Tables, 3 Rows, 3 Columns):\n", array_3d)
```

4. **Write a Python/R program to create a data frame from four given vectors (name, score, attempts, qualify).**

```
import pandas as pd

# Create the data dictionary

data = {

'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',

'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],

'score': [12.5, 9, 16.5, 12, 9, 20, 14.5, 13.5, 8, 19],

'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],

'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes']

}

# Convert to DataFrame

df = pd.DataFrame(data)

print(df)
```

5. **Write a Python/R program to create a factor corresponding to height of women data set, which contains height**

6. **Write Python/R program to Print the Fibonacci Sequence.**

```
def print_fibonacci(n_terms):

a, b = 0, 1

sequence = []

for _ in range(n_terms):

sequence.append(a)

a, b = b, a + b

return sequence

print("Fibonacci Sequence (first 10 terms):", print_fibonacci(10))
```

7. **Write Python/R program to import and export data from csv, excel, txt file.**

```
import pandas as pd

# Create a sample dataframe

df = pd.DataFrame({'ID': [1, 2], 'Name': ['A', 'B']})

# EXPORT

df.to_csv('data.csv', index=False)

df.to_excel('data.xlsx', index=False)

df.to_csv('data.txt', sep='\t', index=False)

print("Data exported successfully.")

# IMPORT

df_imported = pd.read_csv('data.csv')

print("\nImported DataFrame from CSV:\n", df_imported)
```

8. **Write a Python/R program to find the sum of digits of a number reducing it to one digit using repeat loop.**

Part 2: Machine Learning & Data Analytics

LA1 (Q2). Use the Iris dataset and train a Logistic Regression model to classify whether a flower is of species Iris-setosa or not. Evaluate model accuracy, precision, and recall.

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, precision_score, recall_score

# Load data and set target (1 if Setosa, 0 otherwise)

iris = load_iris()

X = iris.data

y = (iris.target == 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=4

# Train and predict

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Precision:", precision_score(y_test, y_pred))

print("Recall:", recall_score(y_test, y_pred)) # Corrected Syntax
```

LA2 (Q3). Load the Titanic dataset, perform basic preprocessing (handle missing values, encode categorical variables), and train a Decision Tree classifier. Plot the confusion matrix.

```
import os, seaborn as sns, pandas as pd, matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
```

LA3 (Q4). **Use the Naive Bayes classifier to perform sentiment analysis on a dataset of movie reviews, classifying them as either positive or negative based on the review text.**

```
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.naive_bayes import MultinomialNB

# Mock dataset

reviews = ["I loved this movie", "Terrible film, hated it", "Great acting and plot", "W

labels = [1, 0, 1, 0] # 1: Positive, 0: Negative

# Vectorize text

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(reviews)

# Train model

nb = MultinomialNB()

nb.fit(X, labels)

# Test model

test_reviews = ["The movie was great", "I hated the plot"]

X_test = vectorizer.transform(test_reviews)

print("Predictions (1=Pos, 0=Neg):", nb.predict(X_test))
```

LA4 (Q6). **A company wants to predict the salary of employees based on their years of experience. Use simple linear regression to model the relationship between salary and experience.**

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

# Generate mock data

np.random.seed(42)
```

LA5 (Q7). Use a dataset with features like square footage vs. house price. Implement a simple linear regression model to predict house prices. Plot the regression line and calculate MSE and R² score.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

# Generate mock data

sq_foot = np.random.uniform(500, 5000, 100).reshape(-1, 1)

price = 50000 + (sq_foot * 150) + np.random.normal(0, 50000, (100, 1))

# Train model

reg = LinearRegression()

reg.fit(sq_foot, price)

y_pred = reg.predict(sq_foot)

print("MSE:", mean_squared_error(price, y_pred))

print("R² Score:", r2_score(price, y_pred))

# Plotting with explicit parameters for clarity

plt.figure(figsize=(8, 6))

plt.scatter(sq_foot, price, color='green', label='Actual House Prices')

plt.plot(sq_foot, y_pred, color='orange', linewidth=2, label='Regression Line')

plt.title("House Price Prediction based on Square Footage")

plt.xlabel("Square Footage")

plt.ylabel("Price (USD)")

plt.grid(True, linestyle='--', alpha=0.6)
```

LA7 & LA8 (Q9 & Q10). **Using a grocery dataset with minimum support to 0.001 and minimum confidence of 0.8 build a frequent pattern tree (FP-Tree) and use the Apriori algorithm to indicate the top 5 association rules.**

```
import pandas as pd

import warnings

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

# Suppress Jupyter runtime/deprecation warnings

warnings.filterwarnings('ignore')

# Mock transactions (Self-contained)

dataset = [

['Milk', 'Onion', 'Nutmeg', 'Eggs', 'Yogurt'],

['Dill', 'Onion', 'Nutmeg', 'Eggs', 'Yogurt'],

['Milk', 'Apple', 'Eggs'],

['Corn', 'Onion', 'Ice cream', 'Eggs']

]

te = TransactionEncoder()

te_ary = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_ary, columns=te.columns_)

# FP-Growth (Q9 equivalent)

freq_items = fpgrowth(df, min_support=0.2, use_colnames=True)

print("FP-Growth Frequent Items:\n", freq_items.head(3), "\n")

# Apriori (Q10)

freq_ap = apriori(df, min_support=0.2, use_colnames=True)
```

LA10 (Q12). Create Bar Chart-Stacked taking x axis as Month and Y axis as Revenue and add Legend.

```
import pandas as pd

import matplotlib.pyplot as plt

months = ["Mar", "Apr", "May", "Jun", "Jul"]

colors = ["green", "orange", "brown"]

revenue = {

'East': [100, 150, 200, 250, 300],

'West': [50, 100, 150, 200, 250],

'North': [80, 120, 180, 220, 280]

}

df = pd.DataFrame(revenue, index=months)

df.plot(kind='bar', stacked=True, color=colors, figsize=(8, 5))

plt.title("Revenue by Month and Region")

plt.xlabel("Month")

plt.ylabel("Revenue")

plt.legend(title="Regions")

plt.show()
```

LA11 (Q13). Read the file moviesData.csv to create a bar chart of critics_score for the first 10 movies. Save the plot.

```
import os

import pandas as pd

import matplotlib.pyplot as plt

import numpy as np
```

LA12 (Q14). Use the data set "mtcars" and create boxplot for "mpg" and "cyl" columns.

```
import seaborn as sns

import matplotlib.pyplot as plt

# Load seaborn's equivalent of mtcars

mtcars = sns.load_dataset('mpg').dropna()

plt.figure(figsize=(8, 5))

sns.boxplot(x='cylinders', y='mpg', data=mtcars, palette='Set2')

plt.title("Boxplot of MPG by Cylinders")

plt.xlabel("Cylinders")

plt.ylabel("MPG")

plt.show()
```